



Sistemas Digitais Avançados

Prof. Imbiriba

Prática ULA FULL – Unidade Lógica e Aritmética Completa de 8 bits

Uma Unidade Lógica Aritmética (ULA) é um componente fundamental em qualquer processador de computador. É responsável por realizar as operações aritméticas e lógicas necessárias para realizar cálculos e manipulações de dados.

A ULA é composta por circuitos digitais que realizam operações básicas, como adição, subtração, multiplicação e divisão, bem como operações lógicas, como AND, OR e NOT. Esses circuitos são projetados para trabalhar com valores binários, que são a base da computação digital.

A ULA é capaz de receber dados de registradores ou da memória principal, executar as operações solicitadas e, em seguida, armazenar o resultado em um registrador ou na memória principal. A ULA é, portanto, uma das partes mais importantes de um processador, pois é responsável por realizar a maior parte das operações necessárias para a execução de programas.

O código apresentado é uma descrição em VHDL de uma Unidade Lógica Aritmética (ULA) de 8 bits. Essa ULA tem duas entradas A e B, que são números inteiros com valores entre 0 e 255. Também possui uma entrada operação, que é um vetor lógico de 3 bits que especifica a operação a ser realizada. As saídas da ULA são o resultado da operação (resultadoL) e o bit de overflow (overflow), que indica se houve overflow na operação.

A implementação da ULA é feita em uma arquitetura **Behavioral**, que é uma forma de implementar o comportamento de um circuito digital em VHDL. A ULA é implementada como um processo que é sensível às entradas A, B e operação. O processo calcula o resultado da operação e atualiza as saídas resultadoL, resultadoH e overflow de acordo com o resultado da operação.

O processo usa variáveis internas temp_16bits e SOME_VECTOR para armazenar valores temporários. A variável temp_16bits é um inteiro de 16 bits que é usado para armazenar o resultado da operação aritmética. A variável SOME_VECTOR é um vetor lógico de 16 bits que é usado para converter valores inteiros em valores lógicos e vice-versa.

O código usa a função **to_unsigned** da biblioteca NUMERIC_STD para converter valores inteiros em valores lógicos. A função **to_unsigned** converte um valor inteiro em um valor lógico sem sinal de um determinado comprimento.

Em resumo, o código VHDL implementa uma ULA de 8 bits que pode realizar as operações de soma, subtração, multiplicação, divisão, AND, OR, XOR e NOT. A ULA usa variáveis internas para armazenar valores temporários e usa a função **to_unsigned** para converter valores inteiros em valores lógicos.

PARTE 1

Digite o código a seguir no Quartus II

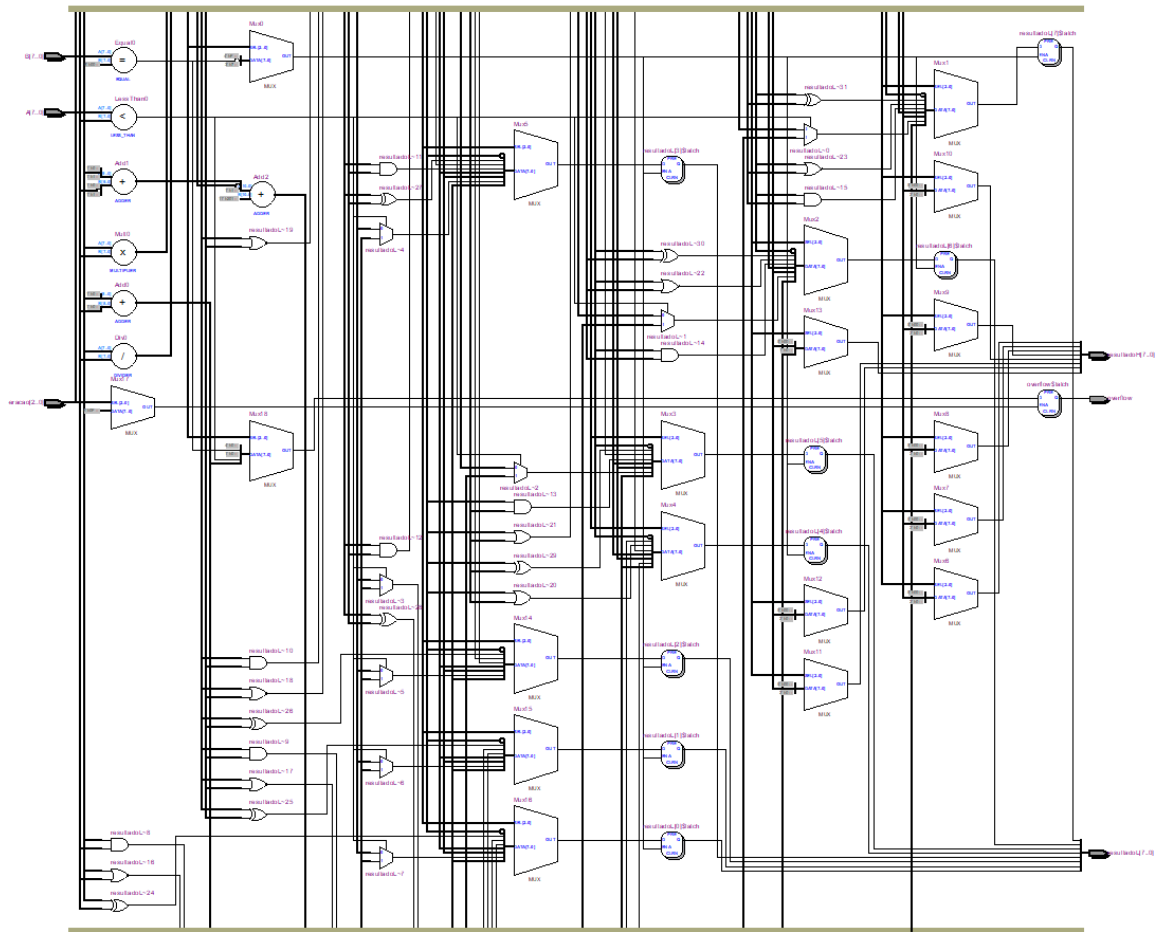


```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity ULA_FULL_8BITS is
6  port (
7      A, B: in integer range 0 to 255;
8      operacao: in std_logic_vector(2 downto 0);
9      resultadoL, resultadoH: out std_logic_vector(7 downto 0);
10     overflow: out std_logic
11 );
12 end ULA_FULL_8BITS;
13
14 architecture Behavioral of ULA_FULL_8BITS is
15 begin
16     process(A, B, operacao)
17         variable temp_16bits: integer range 0 to 65535 := 0;
18         variable SOME_VECTOR : std_logic_vector(15 downto 0) := x"0000" ;
19     begin
20         resultadoH <= x"00";
21         case operacao is
22             when "000" => -- OPERAÇÃO SOMA
23                 temp_16bits := A + B;
24                 SOME_VECTOR := std_logic_vector(to_unsigned(temp_16bits, SOME_VECTOR'length));
25                 resultadoL <= SOME_VECTOR (7 downto 0);
26                 resultadoH <= SOME_VECTOR (15 downto 8);
27                 overflow <= SOME_VECTOR(8);
28
29             when "001" => -- OPERAÇÃO SUBTRAÇÃO
30                 temp_16bits := A - B;
31                 if A < B then
32                     overflow <= '1';
33                     temp_16bits := 256 - temp_16bits;
34                     SOME_VECTOR := std_logic_vector(to_unsigned(temp_16bits, SOME_VECTOR'length));
35                     resultadoL <= SOME_VECTOR (7 downto 0);
36                     resultadoH <= x"00";
37                 else
38                     overflow <= '0';
39                     SOME_VECTOR := std_logic_vector(to_unsigned(temp_16bits, SOME_VECTOR'length));
40                     resultadoL <= SOME_VECTOR (7 downto 0);
41                     resultadoH <= x"00";
42                 end if;
43
44             when "010" => -- OPERAÇÃO MULTIPLICAÇÃO
45                 temp_16bits := A * B;
46                 SOME_VECTOR := std_logic_vector(to_unsigned(temp_16bits, SOME_VECTOR'length));
47                 resultadoL <= SOME_VECTOR (7 downto 0);
48                 resultadoH <= SOME_VECTOR (15 downto 8);
49                 overflow <= '0';
50
51             when "011" => -- OPERAÇÃO DIVISÃO
52                 if B = 0 then
53                     overflow <= '1';
54                 else
55                     temp_16bits := A / B;
56                     overflow <= '0';
57                     SOME_VECTOR := std_logic_vector(to_unsigned(temp_16bits, SOME_VECTOR'length));
58                     resultadoL <= SOME_VECTOR (7 downto 0);
59                     resultadoH <= SOME_VECTOR (15 downto 8);
60                 end if;
61
62             when "100" => -- OPERAÇÃO AND
63                 SOME_VECTOR (7 downto 0) := std_logic_vector(to_unsigned(A, SOME_VECTOR (7 downto 0)'length));
64                 SOME_VECTOR (15 downto 8) := std_logic_vector(to_unsigned(B, SOME_VECTOR (15 downto 8)'length));
65                 resultadoL <= SOME_VECTOR (7 downto 0) and SOME_VECTOR (15 downto 8);
66
67             when "101" => -- OPERAÇÃO OR
68                 SOME_VECTOR (7 downto 0) := std_logic_vector(to_unsigned(A, SOME_VECTOR (7 downto 0)'length));
69                 SOME_VECTOR (15 downto 8) := std_logic_vector(to_unsigned(B, SOME_VECTOR (15 downto 8)'length));
70                 resultadoL <= SOME_VECTOR (7 downto 0) or SOME_VECTOR (15 downto 8);
71
72             when "110" => -- OPERAÇÃO XOR
73                 SOME_VECTOR (7 downto 0) := std_logic_vector(to_unsigned(A, SOME_VECTOR (7 downto 0)'length));
74                 SOME_VECTOR (15 downto 8) := std_logic_vector(to_unsigned(B, SOME_VECTOR (15 downto 8)'length));
75                 resultadoL <= SOME_VECTOR (7 downto 0) xor SOME_VECTOR (15 downto 8);
76
77             when "111" => -- OPERAÇÃO NOT
78                 SOME_VECTOR (7 downto 0) := std_logic_vector(to_unsigned(A, SOME_VECTOR (7 downto 0)'length));
79                 resultadoL <= not SOME_VECTOR (7 downto 0);
80
81             when others =>
82                 temp_16bits := 0;
83                 overflow <= '0';
84         end case;
85     end process;
86 end Behavioral;
```



PARTE 2

Em Tools->Netlist Viewers->RTL Viewer gere o arquivo netlist do projeto da ULA.



PARTE 3

Faça a simulação do circuito usando o UNIVERSITY PROGRAM VWF considerando as formas de onda de entrada conforme mostrado abaixo.

